



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO®



TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE LEÓN

Persistencia en la nube y Dashboard de control

Sistemas Programables
Tapia Ibarra Ma. Veronica

PRESENTA:

Ramírez Abundiz Berenice
Rivera Ponce David Eduardo
Varela Ambriz Saul

León, Guanajuato, Fecha:08 de Junio del 2026



1. ENLACE A REPOSITORIOS

a. ENLACE A REPOSITORIO DE APP MOVIL:

SE UBICA EN LA CARPETA DE
codigo interfaz android/BastonInteligente

<https://github.com/SaulVarelaAmbriz/Baston-Inteligente-MQTT>

b. ENLACE A REPOSITORIO DE SCRIPTS DE PYTHON (FIREBASE):

SE UBICAN EN LA CARPETA DE FIREBASE → FIREBASE.PY

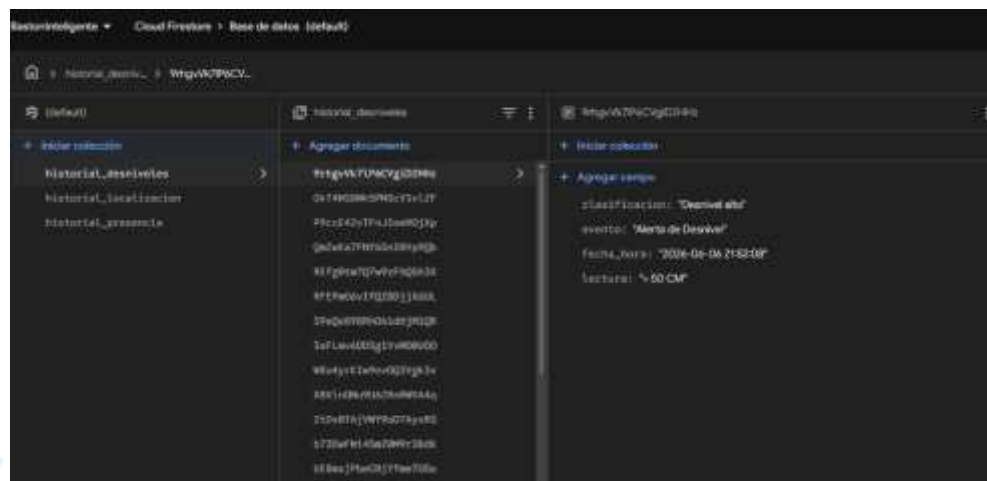
<https://github.com/SaulVarelaAmbriz/Baston-Inteligente-MQTT>

2. BASE DE DATOS EN TIEMPO REAL (REALTIME DATABASE)

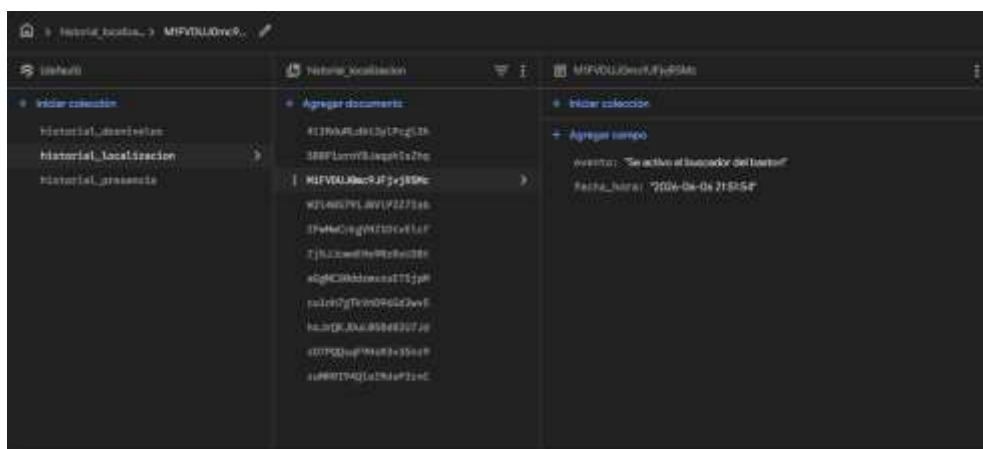


3. EVENTOS CON TIMESTAMP (FIRESTORE)

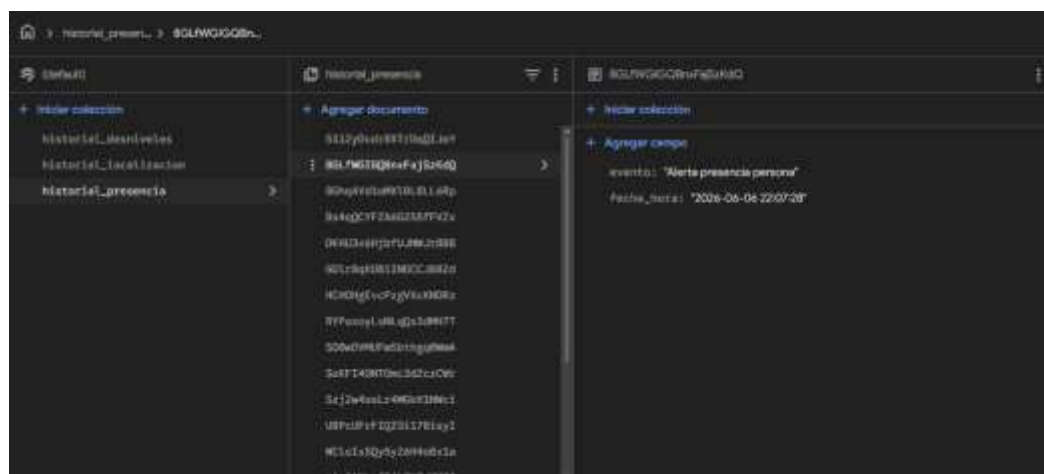
a. HISTORIAL DESNIVELES :



b. HISTORIAL LOCALIZACION DE BASTON:



c. HISTORIAL PRESENCIA DE PERSONA:

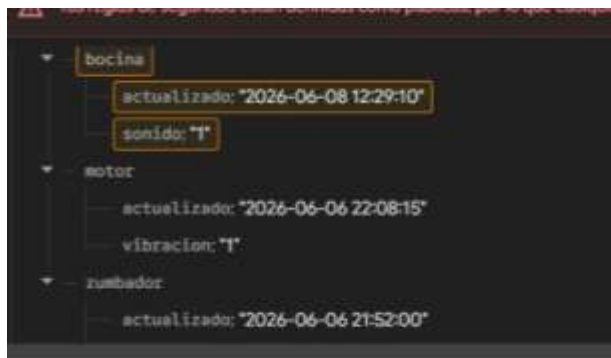


4. DASHBOARD





5. CONTROL REMOTO->ACTUADOR RESPONDIENDO A UNA ORDEN DE LA INTERFAZ



```

Firebase -> Enviados [2026-06-08 12:29:13]: None
[2026-06-08 12:29:13] Telemetría Recibida -> bocina: alto
Firebase -> Enviados [2026-06-08 12:29:13]: alto
Histórico registrado: Desnivel alto (> 50 CM)
[2026-06-08 12:29:14] Telemetría Recibida -> bocina: Sin mensaje
Firebase -> Enviados [2026-06-08 12:29:14]: Sin mensaje
[2026-06-08 12:29:15] Telemetría Recibida -> ultrasonico: 221
Firebase -> Enviados [2026-06-08 12:29:15]: 221
[2026-06-08 12:29:16] Telemetría Recibida -> pir: 1
Firebase -> Enviados [2026-06-08 12:29:16]: 1
[2026-06-08 12:29:17] Telemetría Recibida -> control: None
Firebase -> Enviados [2026-06-08 12:29:17]: None
[2026-06-08 12:29:20] Telemetría Recibida -> ultrasonico: 250
```

6. ANALISIS INDIVIDUAL

a. BERENICE RAMIREZ ABUNDIZ:

i. PROBLEMAS ENCONTRADOS:

1. **Limitaciones de Hardware y Memoria RAM en el Sistema Embebido:** Al intentar implementar la transferencia de datos en tiempo real hacia Firebase directamente desde el módulo de comunicación de la ESP32, se identificó que el protocolo HTTPS y el procesamiento de los paquetes JSON requerían un consumo de memoria RAM elevado. Al ser los recursos del microcontrolador insuficientes para mantener la estabilidad de la HAL y las conexiones de red simultáneamente, el sistema sufría congelamientos.



2. **Restricciones de Autenticación en la Plataforma Cloud:** Se detectó que Cloud Firestore exige de manera estricta mecanismos de autenticación avanzados mediante llaves privadas criptográficas. No era factible utilizar una API Key convencional expuesta directamente en el código de la ESP32 para gestionar bases de datos relacionales o estructuradas en la nube.
3. **Incompatibilidad de Configuración en el Entorno Móvil:** Durante el desarrollo de la aplicación móvil, el entorno de Firebase rechazaba las peticiones de lectura debido a la ausencia de los archivos de configuración y credenciales nativas necesarias para enlazar el proyecto de Kotlin con los servidores de Google.
4. **Persistencia de Datos en Entornos Desconectados (Modo Offline):** Uno de los mayores retos de lógica de software fue diseñar una arquitectura que permitiera almacenar, estructurar y visualizar la información localmente en la aplicación móvil cuando el dispositivo no contara con acceso activo a Internet.

ii. SOLUCIONES APLICADAS:

1. **Diseño de Arquitectura Puente (Servidor Intermedio en PC):** Se optó por delegar el procesamiento pesado y la persistencia en la nube a un servidor intermedio ejecutado en una computadora. Para ello, se modularizó el código creando una clase independiente encargada de suscribirse de forma asíncrona al broker público de MQTT. Este servidor procesa la telemetría del bastón en tiempo real y la deposita simultáneamente en dos plataformas: Realtime Database (para el estado actual) y Cloud Firestore, generando un registro histórico detallado de eventos críticos (detección de desniveles, presencia de personas y localización).
2. **Integración de Credenciales de Administrador con Google Services:** Para resolver los fallos de comunicación y seguridad desde el entorno móvil, se generó e integró el archivo de configuración oficial google-services.json dentro de la raíz del proyecto Android, estableciendo un canal de comunicación autenticado y seguro.



3. **Implementación de Librerías Nativas para Kotlin:** Se realizó una investigación tecnológica para seleccionar e implementar dependencias y librerías nativas de Firebase que fueran completamente compatibles con el lenguaje Kotlin. Esto permitió activar la persistencia local de datos en la aplicación, garantizando que el usuario de apoyo pueda visualizar la bitácora del invidente incluso en situaciones donde no haya cobertura de red.

iii. CONCLUSION:

El desarrollo e integración del proyecto "Bastón Inteligente" permitió demostrar la viabilidad de conjuntar tecnologías de hardware embebido, protocolos de comunicación industriales y servicios en la nube para crear un dispositivo de asistencia médica de alto impacto social.

A lo largo del proyecto, el mayor reto de implementación técnica consistió en el diseño de la lógica de software encargada de interceptar los datos desde el broker MQTT y dispersarlos de manera eficiente hacia el ecosistema de Firebase. Lograr la sincronización paralela entre el broker, Cloud Firestore y Realtime Database fue un proceso crítico para garantizar la reactividad de los actuadores en tiempo real. Gracias a la transición hacia una arquitectura de servidor intermedio (puente de datos) basada en eventos, se eliminaron los cuellos de botella por consumo de memoria RAM en la ESP32. Esto aseguró que la aplicación móvil en Kotlin reciba y visualice las alertas de desniveles, presencia y localización de forma inmediata, cumpliendo con los estrictos criterios de seguridad y usabilidad indispensables para salvaguardar la integridad de los usuarios invidentes.

b. DAVID EDUARDO RIVERA PONCE:

i. PROBLEMAS ENCONTRADOS:

1. **Incompatibilidad de formatos de datos:** Los datos de los sensores que enviaba el bastón por MQTT llegaban como texto simple (strings). Al intentar subirlos directamente a Firestore desde el servidor de Python, la base de datos rechazaba los registros porque exigía un formato estructurado y limpio para poder acomodarlos en el historial.



2. **Saturación en el envío de coordenadas:** Al integrar las funciones del GPS en el servidor de Python, el script intentaba registrar la ubicación en la base de datos de manera fija. Si el bastón mandaba datos corruptos o incompletos por falta de señal, el servidor de Python se detenía por un error de conversión, interrumpiendo el flujo hacia Firebase.

3. **Duplicidad de alertas en el historial:** Cuando un sensor (como el PIR) se activaba y se quedaba en estado positivo por unos segundos, el servidor de Python recibía múltiples mensajes seguidos por MQTT. Esto causaba que se registraran decenas de documentos idénticos en Firestore en un solo segundo, llenando la base de datos de información repetida.

ii. SOLUCIONES APLICADAS:

1. **Estructuración y conversión en Python:** Se programó una función dentro del servidor de Python para limpiar y ordenar los datos recibidos antes de mandarlos a la nube. El texto se convierte en un diccionario organizado (JSON), lo que permite que Firestore reciba la información clasificada por fecha, hora y tipo de evento.

2. **Validación de datos del GPS:** Se agregó un filtro en el código de Python que revisa las coordenadas antes de subirlas a Firebase. Si los datos del GPS vienen vacíos o con errores, el servidor los ignora o registra un valor por defecto en la base de datos, evitando que el programa se detenga.

3. **Filtro de eventos únicos:** Se desarrolló una lógica en el servidor para controlar el flujo de almacenamiento. Ahora, cuando se recibe una alerta, Python verifica que sea un evento nuevo; si es una lectura repetida en un lapso muy corto, no la vuelve a subir a Firestore, manteniendo el historial limpio y ordenado.

iii. CONCLUSION:

La correcta programación del servidor en Python fue una pieza clave para conectar el bastón con los servicios en la nube. El principal desafío en esta sección fue asegurar que los datos rústicos de los sensores se transformaran en información útil y ordenada para las bases de datos de Google. Al diseñar filtros



que limpian los errores del GPS y evitan los registros repetidos en Firestore, logramos que la transferencia de datos sea ligera y precisa. Esto garantiza que la información que consume la aplicación móvil en Kotlin sea totalmente confiable, estructurada y esté lista para ser leída por los familiares del usuario sin retrasos en el Sistema

c. SAUL VARELA AMBRIZ

i. PROBLEMAS ENCONTRADOS

1. **Rechazo de comunicación en Android:** Al intentar conectar la aplicación móvil desarrollada en Kotlin con la Realtime Database, el sistema bloqueaba las peticiones de lectura de datos. Esto ocurría porque las reglas de seguridad de Firebase venían configuradas en modo cerrado por defecto y rechazaban las conexiones de dispositivos externos.
2. **Librerías incompatibles en el entorno móvil:** Durante el diseño de la aplicación en el entorno de desarrollo (Android Studio), se presentaron errores al compilar el código debido a que se utilizaban dependencias de Firebase obsoletas o que no eran totalmente compatibles con la versión actual de Kotlin.
3. **Pantalla congelada por consultas en internet:** Al principio, la aplicación de Kotlin realizaba las peticiones para descargar el historial de Firestore directamente en el hilo principal del teléfono. Cuando la conexión a internet fallaba o era lenta, la pantalla de la aplicación se congelaba por completo y no permitía al usuario presionar ningún botón.

ii. SOLUCIONES

1. **Configuración de reglas de acceso:** Se modificaron los permisos y las reglas de seguridad dentro de las consolas de Realtime Database y Cloud Firestore. Esto abrió las compuertas para que la aplicación en Kotlin pudiera leer la telemetría actual y los historiales de forma remota mediante credenciales seguras.
2. **Actualización del Gradle con dependencias nativas:** Se realizó una limpieza y actualización en los archivos de configuración de la app (build.gradle), reemplazando las



librerías antiguas por las versiones oficiales y nativas de Firebase para Kotlin, logrando una integración limpia y libre de errores de compilación.

3. **Consultas asíncronas en segundo plano:** Se reestructuró el código de Kotlin implementando tareas en segundo plano (Corrutinas). De esta forma, la descarga del historial de desniveles o presencia se procesa de forma oculta en el teléfono; si el internet es lento, la aplicación muestra una animación de carga pero nunca se traba, permitiendo que el usuario siga navegando por el menú.

iii. CONCLUSION

El desarrollo de la aplicación móvil en Kotlin demostró que la experiencia de usuario y la seguridad son fundamentales al conectar un proyecto de IoT con la nube. El mayor reto consistió en enlazar la interfaz móvil con las dos bases de datos de Firebase sin comprometer el rendimiento del teléfono. Al solucionar los errores de permisos, actualizar las dependencias del sistema e implementar consultas en segundo plano, logramos una aplicación fluida y estable. El resultado es una interfaz intuitiva en Kotlin que permite a las personas de apoyo consultar la ubicación, los desniveles y las alertas del bastón en tiempo real, cumpliendo con el objetivo de ser un sistema de monitoreo constante y seguro